

LOAN APPROVAL PREDICTION USING MACHINE LEARNING

(Technical Documentation)

Prepared by : Somoshree Pal

Intern Id: 4821

Internship Domain : Machine Learning

Project Title :

Loan Approval Prediction Using Machine Learning

TABLE OF CONTENTS

1. Project Overview
2. Problem Statement
3. Project Objectives
4. Project Scope
5. Dataset Description
6. Software Requirements
7. Hardware Requirements
8. Project Architecture
9. System Workflow
10. Implementation Details
11. Machine Learning Algorithms
12. Testing & Validation
13. Project Outcomes
14. Folder Structure
15. Limitations
16. Future Enhancements
17. References

CHAPTER 1

PROJECT OVERVIEW

1.1 Overview

The Loan Approval Prediction System is a machine learning application developed to automate the process of predicting whether a loan application should be approved based on applicant information. The system analyzes historical loan records and learns patterns that distinguish approved applications from rejected ones.

The project follows the complete machine learning pipeline, beginning with dataset collection and preprocessing, followed by exploratory data analysis, model training, evaluation, and prediction. The final implementation uses the Random Forest Classifier due to its superior performance among the evaluated models.

The developed system assists financial institutions by reducing manual effort, improving consistency, and providing faster loan approval decisions.

1.2 Features of the Project

- Predicts loan approval status.
- Uses historical applicant information.
- Supports multiple machine learning algorithms.
- Performs automatic preprocessing.
- Generates prediction results with high accuracy.
- Identifies the most influential features affecting approval.

- Provides visual insights through graphs.

CHAPTER 2

PROBLEM STATEMENT

Financial institutions process thousands of loan applications every day. Manual verification is time-consuming and prone to inconsistencies. The objective of this project is to develop a machine learning-based prediction system capable of assisting financial institutions by accurately predicting loan approval decisions using applicant data.

CHAPTER 3

PROJECT OBJECTIVES

Primary Objective

Develop a machine learning model for predicting loan approval decisions.

Secondary Objectives

- Analyze applicant data.
- Clean and preprocess the dataset.
- Train multiple machine learning models.
- Compare model performance.
- Identify key influencing features.
- Improve decision-making efficiency.

CHAPTER 4

PROJECT SCOPE

The project focuses on supervised machine learning techniques for binary classification of loan applications.

The documentation covers:

- Data preprocessing
- Feature encoding
- Model training
- Performance evaluation
- Feature importance
- Prediction

The system is intended as a decision-support tool and does not replace human expertise.

CHAPTER 5

DATASET DESCRIPTION

5.1 Dataset Overview

The dataset used in this project contains historical records of loan applicants and their corresponding loan approval status. Each record consists of demographic, financial, employment, and credit-related information that is used by the machine learning model to predict whether a loan should be approved or rejected.

The dataset serves as the foundation for training and evaluating the machine learning models. Before model development, the dataset was thoroughly analyzed and preprocessed to ensure that it was suitable for classification tasks. This included examining the dataset structure, checking for missing values, encoding categorical variables, and separating the features from the target variable.

The target variable in this project is **Loan Status**, where:

- **1** represents **Loan Approved**
- **0** represents **Loan Rejected**

The remaining columns are treated as independent variables (features) that help the machine learning model make predictions.

5.2 Dataset Features

Feature	Description
Age	Age of the applicant
Gender	Gender of the applicant
Education	Highest educational qualification
Person Income	Annual income of the applicant
Employee Experience	Years of work experience
Home Ownership	Type of home ownership
Loan Amount	Amount requested as loan
Loan Intent	Purpose of taking the loan
Loan Interest Rate	Interest rate charged on the loan
Loan Percentage	Ratio of loan amount to applicant's income
Credit History	Credit history length or category
Credit Score	Credit score of the applicant
Previous Loan	Previous loan history of the applicant
Loan Status	Loan approval status (Target Variable)

5.3 Dataset Exploration

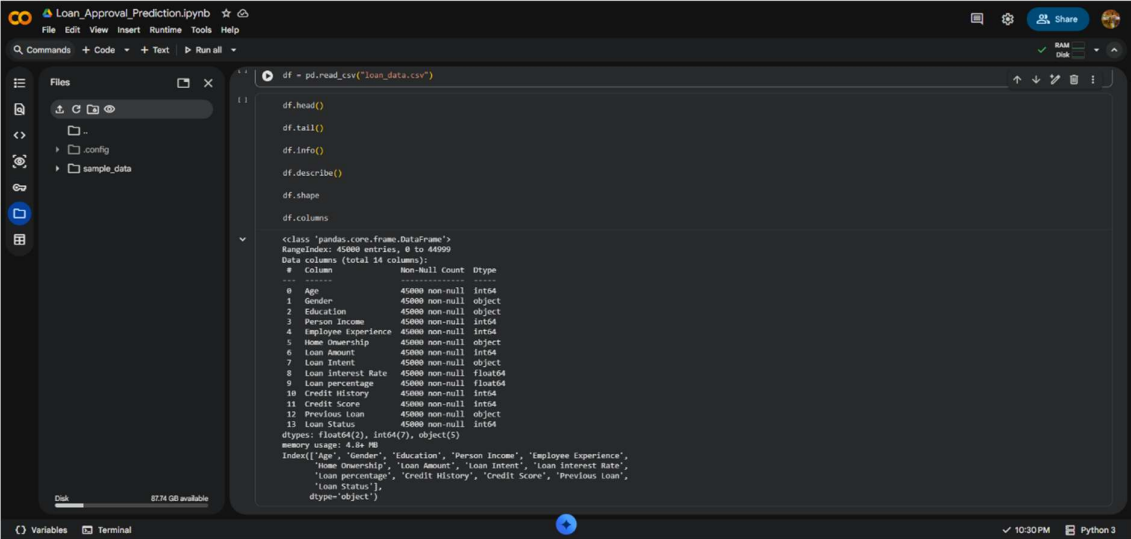
To understand the structure and quality of the dataset, several exploratory functions provided by the Pandas library were used. These functions help in obtaining information such as the first and

last few records, the total number of rows and columns, data types of each feature, statistical summaries, and column names.

The following functions were used during dataset exploration:

- `df.head()`
- `df.tail()`
- `df.info()`
- `df.describe()`
- `df.shape`
- `df.columns`

These functions provided valuable insights into the dataset and helped verify that the data was correctly loaded before proceeding to preprocessing.



```
df = pd.read_csv("loan_data.csv")

df.head()

df.tail()

df.info()

df.describe()

df.shape

df.columns

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 45000 entries, 0 to 44999
Data columns (total 14 columns):
 #   Column             Non-Null Count  Dtype  
---  --
 0   Age                45000 non-null  int64  
 1   Gender              45000 non-null  object  
 2   Education           45000 non-null  object  
 3   Person Income       45000 non-null  int64  
 4   Employee Experience  45000 non-null  int64  
 5   Home Ownership      45000 non-null  object  
 6   Loan Amount         45000 non-null  int64  
 7   Loan Intent         45000 non-null  object  
 8   Loan Interest Rate  45000 non-null  float64  
 9   Loan Percentage     45000 non-null  float64  
10   Credit History      45000 non-null  int64  
11   Credit Score        45000 non-null  int64  
12   Previous Loan       45000 non-null  object  
13   Loan Status         45000 non-null  int64  
dtypes: float64(2), int64(7), object(5)
memory usage: 4.1+ MB

Index(['Age', 'Gender', 'Education', 'Person Income', 'Employee Experience',
       'Home Ownership', 'Loan Amount', 'Loan Intent', 'Loan Interest Rate',
       'Loan Percentage', 'Credit History', 'Credit Score', 'Previous Loan',
       'Loan Status'],
      dtype='object')
```

Figure 5.1: Initial Exploration of the Dataset

CHAPTER 6

SOFTWARE AND HARDWARE REQUIREMENTS

6.1 Software Requirements

The following software tools were used during the development of this project.

Software	Purpose
Google Colab	Development and execution of Python code
Python	Programming language
Google Drive	Dataset storage and backup
Microsoft Word	Report and documentation preparation

6.2 Python Libraries

Several Python libraries were used for data analysis, visualization, and machine learning.

Library	Purpose
Pandas	Data loading and manipulation
NumPy	Numerical computations
Matplotlib	Plot generation
Seaborn	Statistical visualization
Scikit-learn	Machine learning algorithms
Joblib	Saving trained machine learning model

6.3 Hardware Requirements

The project can be executed on a standard personal computer with the following minimum specifications.

- Processor: Intel Core i3 or higher
- RAM: 4 GB minimum (8 GB recommended)
- Storage: 2 GB free disk space
- Internet connection
- Google Chrome browser

CHAPTER 7

PROJECT ARCHITECTURE

7.1 System Architecture

The Loan Approval Prediction System follows a modular architecture where the input data passes through multiple processing stages before generating the final prediction.

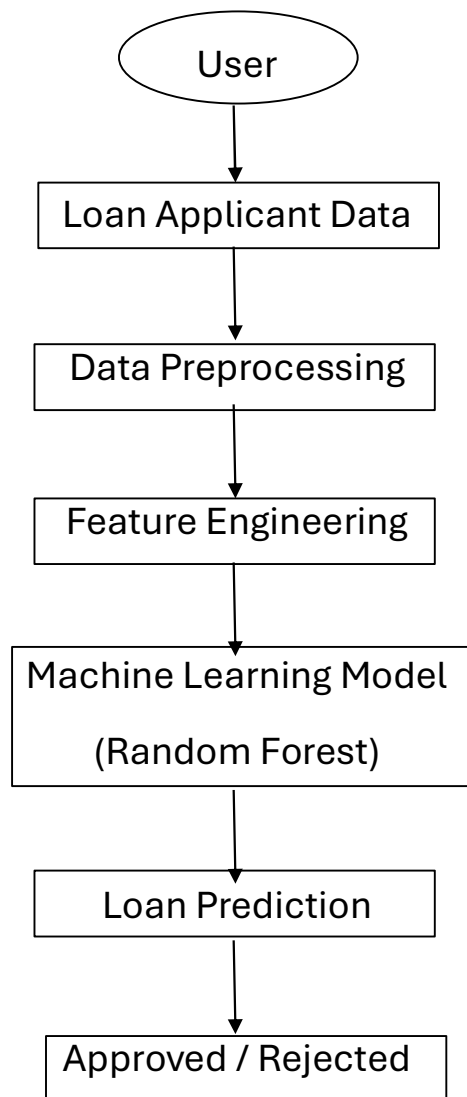


Figure 7.1: System Architecture

7.2 Architecture Explanation

The architecture begins with the collection of applicant information, including demographic, financial, employment, and credit-related details. The input data is then preprocessed by handling missing values and converting categorical variables into numerical form using Label Encoding.

After preprocessing, the prepared dataset is supplied to the machine learning model. Three different classification algorithms were trained during development, namely Logistic Regression, Decision Tree, and Random Forest. Based on the evaluation results, the Random Forest model was selected because it achieved the highest prediction accuracy.

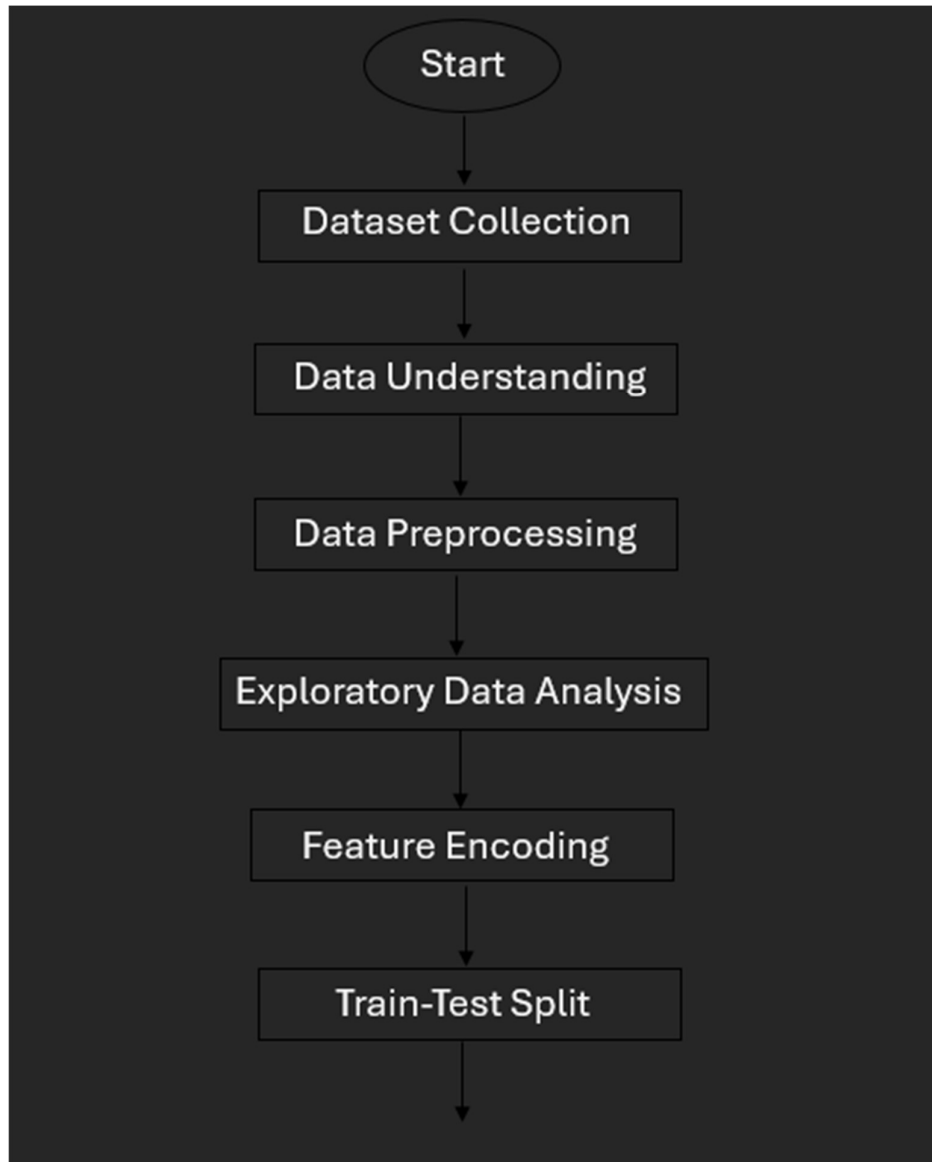
The trained model finally predicts whether the applicant's loan should be approved or rejected based on the provided information.

CHAPTER 8

SYSTEM WORKFLOW

8.1 Workflow

The overall workflow followed during the project development is illustrated below.



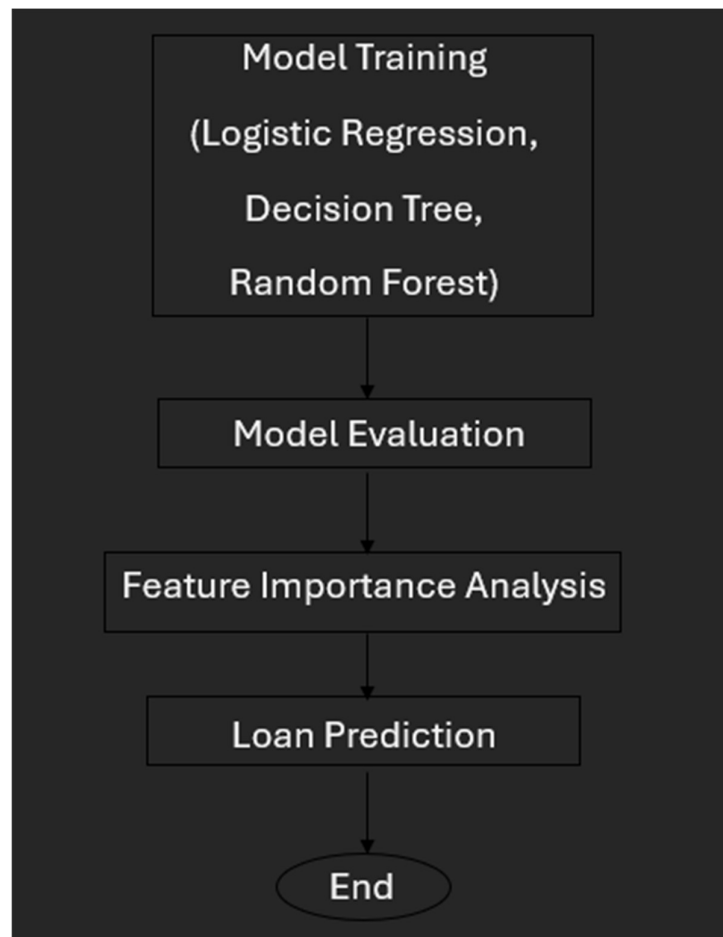


Figure 8.1: Project Workflow

8.2 Workflow Explanation

The workflow begins with collecting the historical loan dataset. The dataset is first analyzed to understand its structure and identify any inconsistencies.

The next stage involves preprocessing the data by checking for missing values, encoding categorical variables, and separating the features from the target variable.

The processed dataset is then divided into training and testing sets using an 80:20 ratio. Three different machine learning algorithms are trained using the training dataset.

The trained models are evaluated using multiple performance metrics, and the best-performing model is selected. Finally, the Random Forest model is used to predict loan approval decisions for new applicants.

CHAPTER 9

IMPLEMENTATION DETAILS

9.1 Dataset Loading

The dataset was imported into Google Colab using the Pandas library.

```
df = pd.read_csv("loan_data.csv")
```

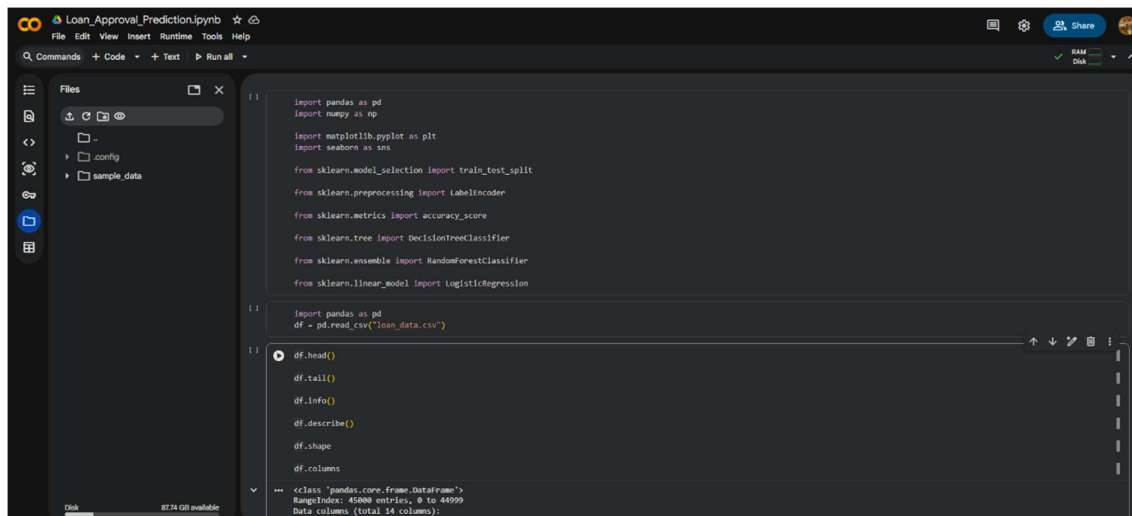


Figure 9.1: Loading Dataset into Google Colab

9.2 Data Exploration

The structure of the dataset was analyzed using the following functions:

- `head()`
- `tail()`
- `info()`
- `describe()`

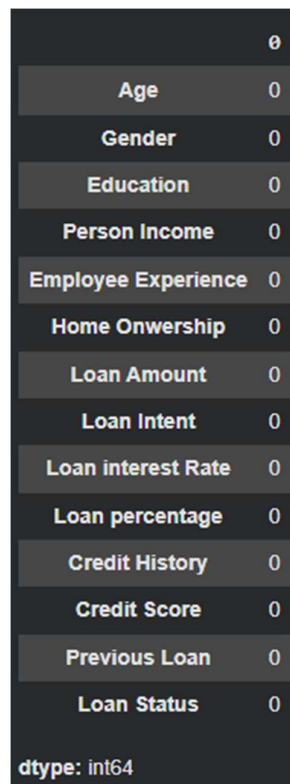
These functions provide information regarding the dataset structure, data types, statistical summaries, and sample records.

9.3 Handling Missing Values

Missing values were checked using the following command:

```
df.isnull().sum()
```

The dataset was inspected to ensure that no missing values remained before training the machine learning models. This step is essential because missing values can negatively affect model performance and prediction accuracy.



	0
Age	0
Gender	0
Education	0
Person Income	0
Employee Experience	0
Home Onwership	0
Loan Amount	0
Loan Intent	0
Loan interest Rate	0
Loan percentage	0
Credit History	0
Credit Score	0
Previous Loan	0
Loan Status	0
dtype: int64	

Figure 9.2: Missing Value Analysis

9.4 Encoding Categorical Variables

Machine learning algorithms require numerical input data. Therefore, categorical variables such as Gender, Education, Home

Ownership, Loan Intent, and Loan Status were converted into numerical values using the Label Encoder technique provided by the Scikit-learn library.

This transformation ensured that the machine learning algorithms could process the dataset efficiently without losing the relationships among categorical values.

	Age	Gender	Education	Person Income	Employee Experience	Home Ownership	Loan Amount	Loan Intent	Loan interest Rate	Loan percentage	Credit History	Credit Score	Previous Loan	Loan Status
0	22	0	4	71948	0	3	35000	4	16.02	0.49	3	561	0	1
1	21	0	3	12282	0	2	1000	1	11.14	0.08	2	504	1	0
2	25	0	3	12438	3	0	5500	3	12.87	0.44	3	635	0	1
3	23	0	1	79753	0	3	35000	3	15.23	0.44	2	675	0	1
4	24	1	4	66135	1	3	35000	3	14.27	0.53	4	586	0	1

Figure 9.3: Dataset After Label Encoding

9.5 Feature Selection

The dataset was divided into independent variables (features) and the dependent variable (target). The independent variables contain applicant information, whereas the dependent variable represents the loan approval status.

```
X = df.drop("Loan Status", axis=1)
```

```
y = df["Loan Status"]
```

9.6 Train-Test Split

The processed dataset was divided into training and testing datasets using an 80:20 ratio.

```
train_test_split(
```

```
    X,
```

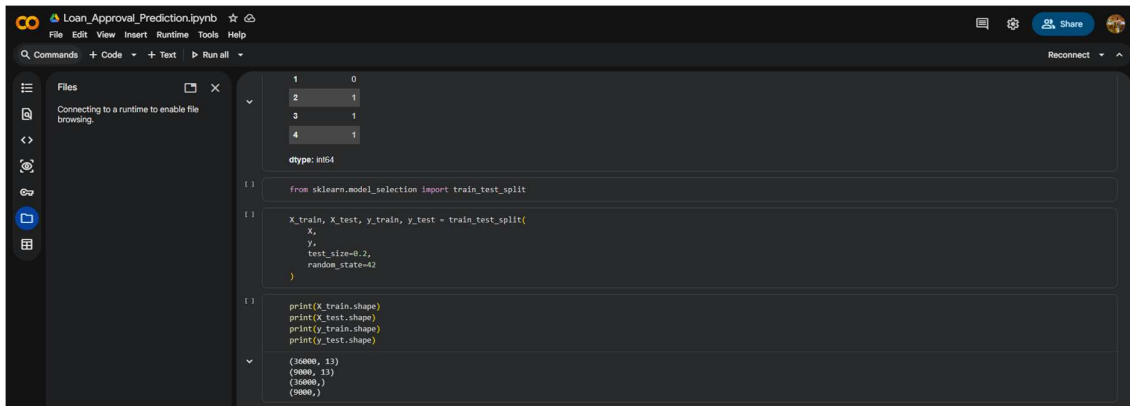
```
    y,
```

```
    test_size=0.2,
```

random_state=42

)

The training dataset was used to train the machine learning models, while the testing dataset was used to evaluate their performance on unseen data.



The screenshot shows a Jupyter Notebook interface with a dark theme. The top bar includes the file name 'Loan_Approval_Prediction.ipynb' and standard menu options. The left sidebar shows a 'Files' panel with a message 'Connecting to a runtime to enable file browsing.' The main area displays a code cell with the following Python code:

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=42
)

print(X_train.shape)
print(X_test.shape)
print(y_train.shape)
print(y_test.shape)
```

Below the code, the output of the print statements is shown:

```
(16000, 11)
(5000, 11)
(16000,)
(5000,)
```

Figure 9.4: Train-Test Split of the Dataset

CHAPTER 10

MACHINE LEARNING ALGORITHMS

10.1 Introduction

Machine Learning algorithms are computational models that learn patterns from historical data and use these learned patterns to make predictions on new or unseen data. Since the objective of this project is to predict whether a loan application should be approved or rejected, it is formulated as a binary classification problem.

To identify the most suitable algorithm for this task, three supervised machine learning classification algorithms were implemented and evaluated. These algorithms are Logistic Regression, Decision Tree Classifier, and Random Forest Classifier. Their performances were compared based on prediction accuracy and other evaluation metrics.

10.2 Logistic Regression

Logistic Regression is one of the most widely used supervised learning algorithms for binary classification problems. Unlike linear regression, which predicts continuous values, Logistic Regression predicts the probability that a given input belongs to a particular class. The algorithm applies the sigmoid function to map the predicted values between 0 and 1, making it suitable for classification tasks.

In this project, Logistic Regression was used as the baseline model for loan approval prediction. The model was trained using the processed training dataset and later evaluated using the testing dataset.

Advantages

- Simple and easy to implement.
- Fast training and prediction.
- Suitable for binary classification problems.
- Produces interpretable results.

Limitations

- Performs poorly on highly complex datasets.
- Assumes a linear relationship between variables.
- Less effective when data contains complex interactions.

```
[ ] from sklearn.linear_model import LogisticRegression
[ ]
[ ] model = LogisticRegression()
[ ]
[ ] model.fit(X_train, y_train)
[ ] /usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge (status=1):
[ ] STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.
[ ]
[ ] Increase the number of iterations (max_iter) or scale the data as shown in:
[ ] https://scikit-learn.org/stable/modules/preprocessing.html
[ ] Please also refer to the documentation for alternative solver options:
[ ] https://scikit-learn.org/stable/modules/linear\_model.html#logistic-regression
[ ] n_iter_i = check_optimize_result(
[ ]     LogisticRegression
[ ]     LogisticRegression()
[ ]
[ ]
[ ] y_pred = model.predict(X_test)
[ ]
[ ] from sklearn.metrics import accuracy_score
[ ]
[ ] accuracy = accuracy_score(y_test, y_pred)
[ ]
[ ] print("Accuracy:", accuracy)
[ ]
[ ] Accuracy: 0.8273333333333334
```

Figure 10.1: Logistic Regression Model Training

10.3 Decision Tree Classifier

Decision Tree is a supervised machine learning algorithm that predicts outcomes by recursively splitting the dataset into smaller subsets based on feature values. Each internal node represents a decision based on a feature, while each leaf node represents the final prediction.

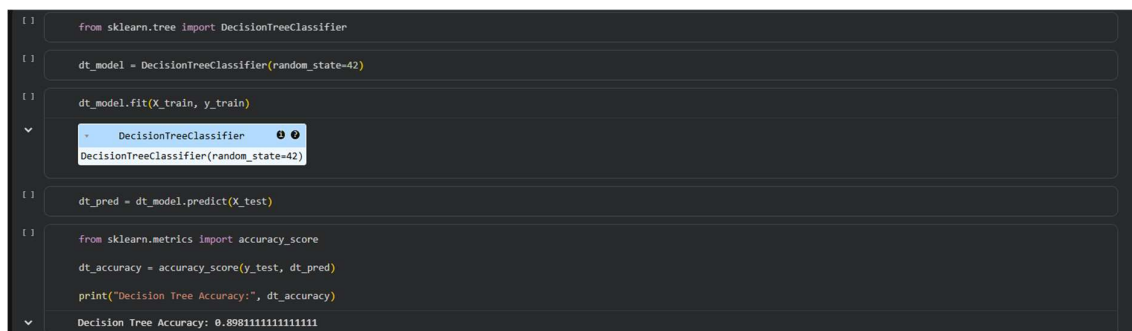
The Decision Tree model was implemented to compare its performance with Logistic Regression and Random Forest. It provided better interpretability but was comparatively more prone to overfitting.

Advantages

- Easy to understand and visualize.
- Handles both numerical and categorical data.
- Does not require feature scaling.

Limitations

- Prone to overfitting.
- Sensitive to small changes in data.
- May produce unstable predictions.



```
[ ] from sklearn.tree import DecisionTreeClassifier
[ ] dt_model = DecisionTreeClassifier(random_state=42)
[ ] dt_model.fit(X_train, y_train)
[ ] dt_model
[ ] dt_pred = dt_model.predict(X_test)
[ ] from sklearn.metrics import accuracy_score
[ ] dt_accuracy = accuracy_score(y_test, dt_pred)
[ ] print("Decision Tree Accuracy:", dt_accuracy)
[ ] Decision Tree Accuracy: 0.8981111111111111
```

Figure 10.2: Decision Tree Model Training

10.4 Random Forest Classifier

Random Forest is an ensemble learning algorithm that combines the predictions of multiple Decision Trees to produce a more accurate and stable result. Instead of relying on a single decision tree, Random Forest constructs numerous trees during training and determines the final prediction through majority voting.

This approach significantly reduces overfitting and improves generalization performance. In this project, the Random Forest classifier achieved the highest prediction accuracy and was selected as the final model.

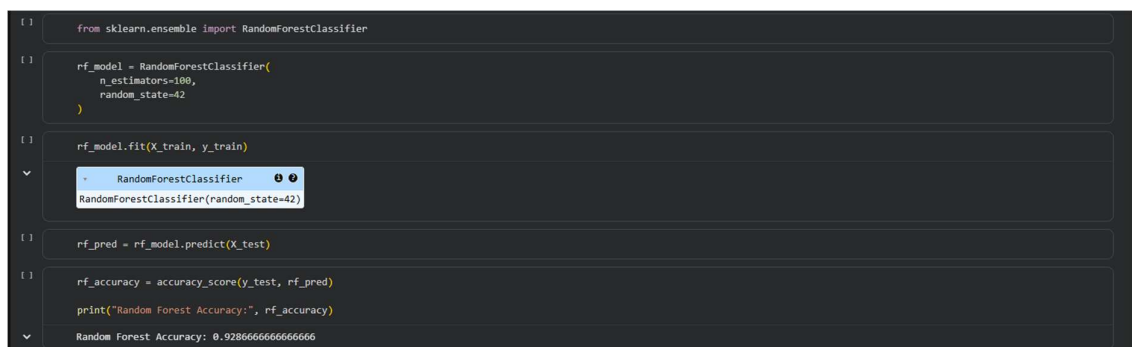
The model successfully captured complex relationships among applicant features and demonstrated superior performance compared to the other implemented algorithms.

Advantages

- High prediction accuracy.
- Robust against overfitting.
- Handles large datasets efficiently.
- Determines feature importance automatically.
- Performs well on both numerical and categorical data.

Limitations

- Higher computational cost.
- More difficult to interpret than a single Decision Tree.



```
[ ] from sklearn.ensemble import RandomForestClassifier

[ ] rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

[ ] rf_model.fit(X_train, y_train)

[ ] rf_pred = rf_model.predict(X_test)

[ ] rf_accuracy = accuracy_score(y_test, rf_pred)
print("Random Forest Accuracy:", rf_accuracy)

Random Forest Accuracy: 0.9286666666666666
```

Figure 10.3: Random Forest Model Training

CHAPTER 11

TESTING AND VALIDATION

11.1 Introduction

After training the machine learning models, their performances were evaluated using the testing dataset. Various evaluation metrics were used to measure the predictive capability of each model and identify the best-performing algorithm.

The evaluation process ensures that the developed model performs accurately on unseen data and can be reliably used for loan approval prediction.

11.2 Accuracy Comparison

The performance of the three implemented machine learning algorithms was compared using prediction accuracy.

Table 11.1: Model Comparison

Model	Accuracy
Logistic Regression	82.70%
Decision Tree	89.81%
Random Forest	92.87%

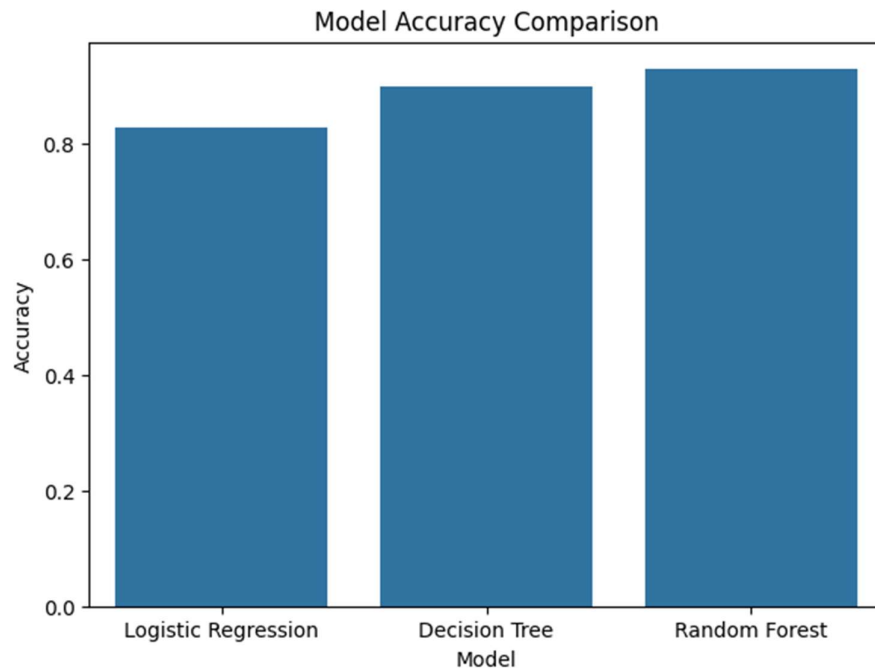


Figure 11.1: Accuracy Comparison of Machine Learning Models

Interpretation

The comparison demonstrates that the Random Forest classifier achieved the highest prediction accuracy among all the implemented algorithms. This superior performance can be attributed to its ensemble learning approach, which combines the predictions of multiple decision trees and minimizes overfitting. Consequently, the Random Forest model was selected as the final prediction model for this project.

11.3 Confusion Matrix

A confusion matrix provides a detailed summary of classification results by comparing actual class labels with predicted class labels. It helps identify correctly classified instances as well as misclassified instances.

The confusion matrix generated for the Random Forest model indicates that the majority of loan applications were classified correctly, demonstrating the effectiveness of the developed model.

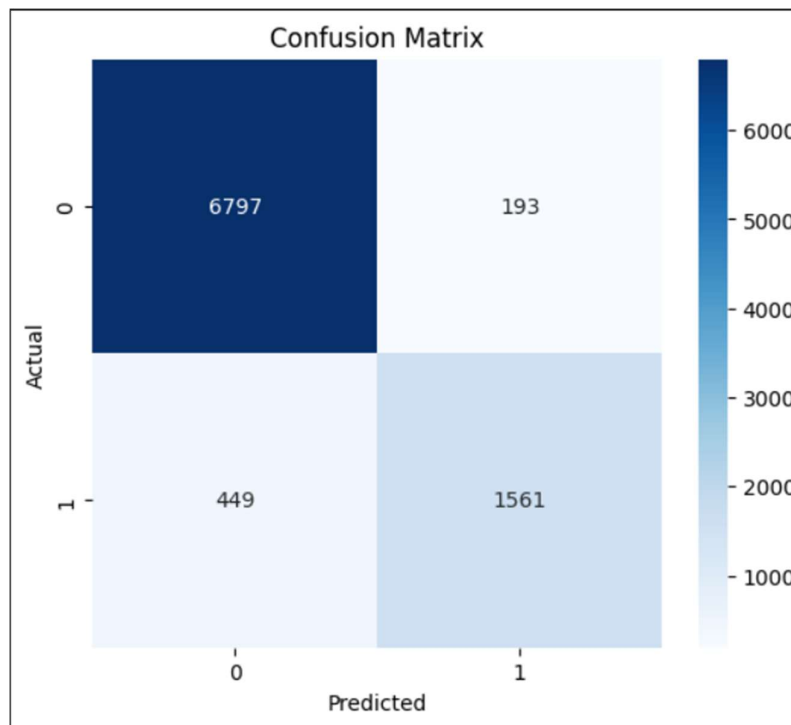


Figure 11.2: Confusion Matrix of the Random Forest Model

11.4 Classification Report

The classification report provides a detailed evaluation of the model using Precision, Recall, F1-Score, and Support.

- **Precision** measures the proportion of correctly predicted positive observations.
- **Recall** measures the proportion of actual positive observations correctly identified.
- **F1-Score** represents the harmonic mean of Precision and Recall.
- **Support** indicates the number of actual observations for each class.

The classification report confirms that the Random Forest model performs consistently across both loan approval and rejection classes.

	precision	recall	f1-score	support
0	0.94	0.97	0.95	6990
1	0.89	0.78	0.83	2010
accuracy			0.93	9000
macro avg	0.91	0.87	0.89	9000
weighted avg	0.93	0.93	0.93	9000

Figure 11.3: Classification Report of Random Forest Model

CHAPTER 12

PROJECT OUTCOMES

12.1 Outcomes Achieved

The Loan Approval Prediction project successfully achieved all the objectives defined at the beginning of the internship.

The major outcomes of the project are summarized below:

- Developed a complete machine learning pipeline for loan approval prediction.
- Successfully preprocessed the dataset by handling missing values and encoding categorical variables.
- Performed exploratory data analysis to understand applicant characteristics.
- Implemented three machine learning classification algorithms.
- Achieved **92.87% prediction accuracy** using the Random Forest classifier.
- Identified the most influential features affecting loan approval decisions.
- Generated multiple visualization graphs for better interpretation of the dataset.
- Developed a reusable machine learning workflow suitable for future improvements.

12.2 Objectives Achieved

Project Objective	Status
Develop ML model	Completed
Achieve >85% accuracy	Achieved (92.87%)
Identify important features	Completed
Analyze historical data	Completed
Improve loan approval efficiency	Achieved through automation

CHAPTER 13

PROJECT FOLDER STRUCTURE

The project files were organized systematically to improve maintainability and simplify future enhancements.

Loan_Approval_Prediction/

├─ Dataset/

| └─ loan_data.csv

|

├─ Images/

| └─ feature_importance.png

| └─ confusion_matrix.png

| └─ correlation_heatmap.png

| └─ accuracy_comparison.png

|

├─ Notebook/

| └─ Loan_Approval_Prediction.ipynb

|

├─ Documentation/

| └─ Project_Documentation.docx

|

├─ Report/

| └─ Loan_Approval_Report.docx


```
|  
└─ Model/  
    └─ loan_approval_model.pkl
```

Folder Description

Dataset: Contains the loan application dataset used for model training.

Images: Contains all graphs, charts, and screenshots used in the report and documentation.

Notebook: Stores the Google Colab notebook containing the complete implementation.

Documentation: Contains the technical documentation of the project.

Report: Contains the detailed project report submitted during the internship.

Model: Stores the trained Random Forest model saved using Joblib.

CHAPTER 14

LIMITATIONS

Although the developed system achieved high prediction accuracy, certain limitations remain.

- The model is dependent on the quality of the available dataset.
- The prediction accuracy may decrease when evaluated on completely different datasets.
- The system does not incorporate real-time financial information.
- Hyperparameter tuning was not extensively performed.
- External economic factors such as inflation and market conditions are not considered.
- The model should be periodically retrained to maintain its effectiveness with new data.

CHAPTER 15

FUTURE ENHANCEMENTS

The developed system provides a strong foundation for future improvements. Several enhancements can be implemented to increase its functionality and performance.

- Develop a user-friendly web application using Streamlit or Flask.
- Integrate the model with real-time banking databases.
- Apply Explainable Artificial Intelligence (XAI) techniques such as SHAP or LIME for better model interpretability.
- Perform hyperparameter optimization to further improve prediction accuracy.
- Evaluate advanced ensemble learning algorithms such as XGBoost, LightGBM, and CatBoost.
- Deploy the model on cloud platforms for real-time access.
- Expand the dataset to include additional applicant attributes such as debt-to-income ratio, repayment history, and employment stability.

CHAPTER 16

REFERENCES

1. Géron, A., *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*, O'Reilly Media, 2019.
2. James, G., Witten, D., Hastie, T., & Tibshirani, R., *An Introduction to Statistical Learning*, Springer, 2021.
3. Pedregosa, F., et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, 2011.
4. McKinney, W., *Python for Data Analysis*, O'Reilly Media, 2022.
5. Hunter, J. D., "Matplotlib: A 2D Graphics Environment," *Computing in Science & Engineering*, 2007.
6. Waskom, M., *Seaborn: Statistical Data Visualization*, 2021.
7. Documentation of Python Libraries:
 - Pandas
 - NumPy
 - Matplotlib
 - Seaborn
 - Scikit-learn